CSF205:
Web Service Development

Week 2: Concurrency and Multithreading

Swansea University
Prifysgol Abertawe

Department of Computer Science
Applied Software Engineering

NOTICE!



Look out for this on slides!
It marks the content as contextual and **NOT** examinable content!



DISCLAIMER

This lecture is full of black magic and lies.

… mostly black magic.



A Foreword: Concurrency

Effectively utilising computing power is an essential role of a programmer.

Modern computers are equipped CPUs capable of managing *many* "tasks" at the same time.

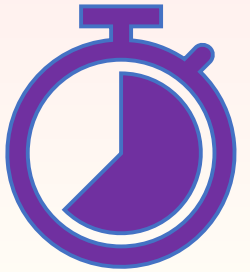
This is called concurrency (not to be confused with parallelism)

Much to my chagrin, we don't dedicate a module to studying it.

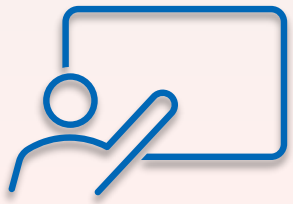
It is covered a bit in your third year (CSF3707), but...

As we're learning about client-server interaction, it makes sense for us to be aware of some of the key concepts.

Note: Not all problems in concurrency are the same. The field is much broader than what we'll consider in this module.



Learning Outcomes



Lecture

- CPUs, Processes and Threads
- Concurrency basics and pitfalls
- Java implementation of multithreading and synchronising code

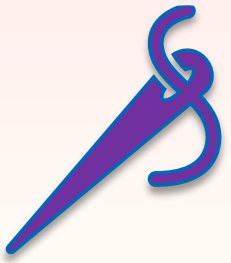


Lab

- Writing your first multithreaded program in Java
- Understanding how to safely synchronise code
- Designing a server capable of supporting multiple client requests

PART 1

CPUs, Threads and Concurrency

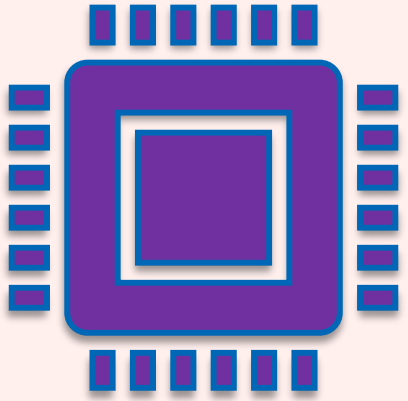


CPUs and Threads: The CPU

Primary component of a computer – often referred to as the 'brain'

It handles the execution of programs:

- Memory management
- I/O
- Arithmetic
- Task Sequencing



…

It executes millions (if not billions) of instructions every second.



CPUs and Threads: CPU Cores

A CPU core is an individual processing unit within the CPU that independently executes tasks.

- Early CPUs were single-core, only capable of handling one task at a time.
- Modern CPUs are multi-core for handling multiple tasks simultaneously.

Each core operates like an independent CPU, capable of executing its own thread of instructions.

They also share resources like cache memory and the connection to the memory controller.

Beyond: You might come across "Simultaneous Multi-Threading" and "Logical Cores" – these are effectively ways for a CPU to "double" the number of threads a core can control.



CPUs and Threads: Workers

- CPU cores are responsible for managing "work".
- "Work" in this context, is just the execution of instructions.
- The CPU schedules this work in a queue.
- That queue is pulled from by separate workers.
- A worker is an independent execution unit that can run in parallel with other workers.
- The "technical" nature of a worker changes depending on the context (operating systems, programming languages, frameworks)

CPUs and Threads: The Thread

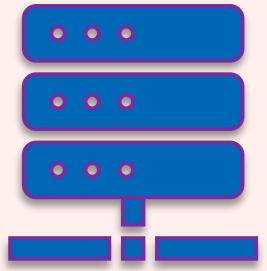
- As far as we, and the OS, are concerned, these workers are implemented as threads.
- Threads are the smallest unit of execution that can be scheduled by an OS or runtime environment.
- Processes are "larger" collections of work and can be made up of different threads.
- The programs we have been writing are single-threaded.
- That meant any 'thread blocking' methods we called from the server socket 'stopped' our server program until a request came from the client.
- What does that mean for us?
- Basically, we can’t deal with more than one client request at a time.

Concurrency: Client Requests

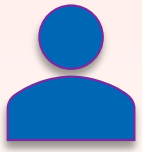
Think back to the simple server sockets from last week.

They were single-threaded and could only deal with one request at a time.

Server



Waits for connection



Client A

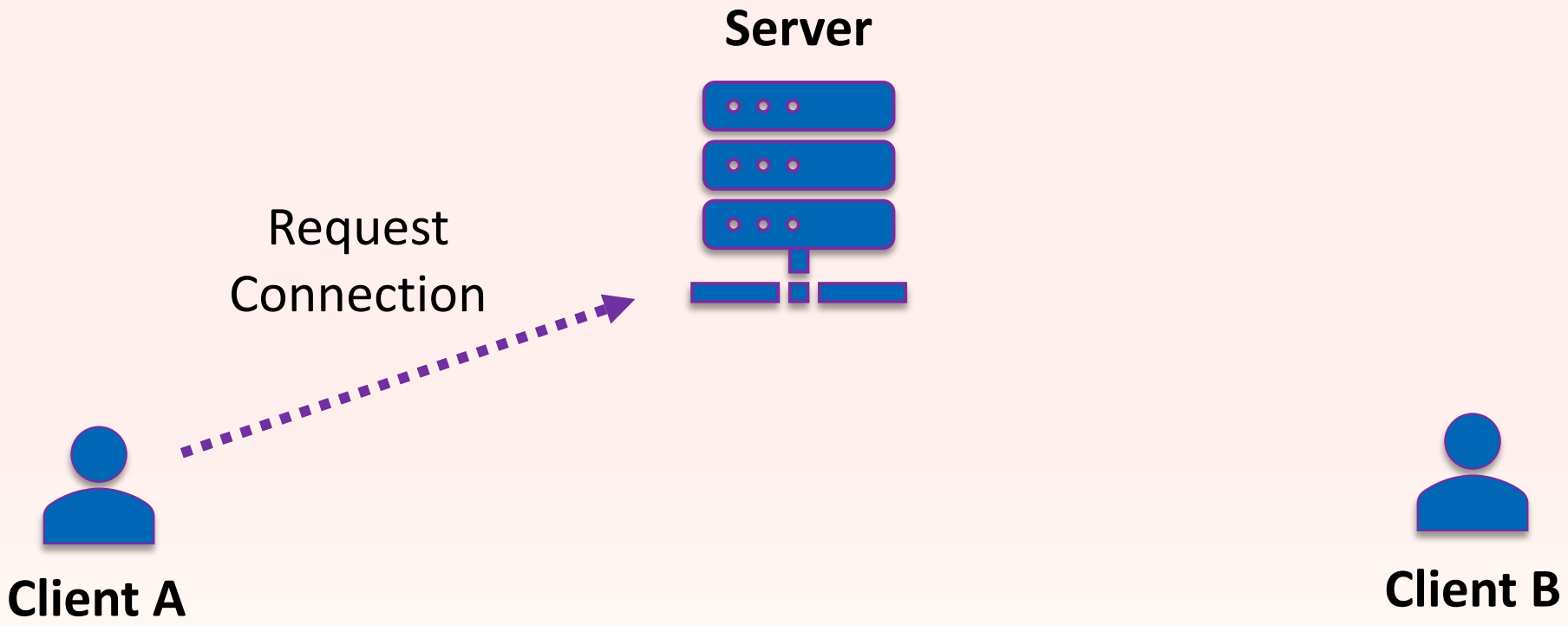


Client B

Concurrency: Client Requests

Think back to the simple server sockets from last week.

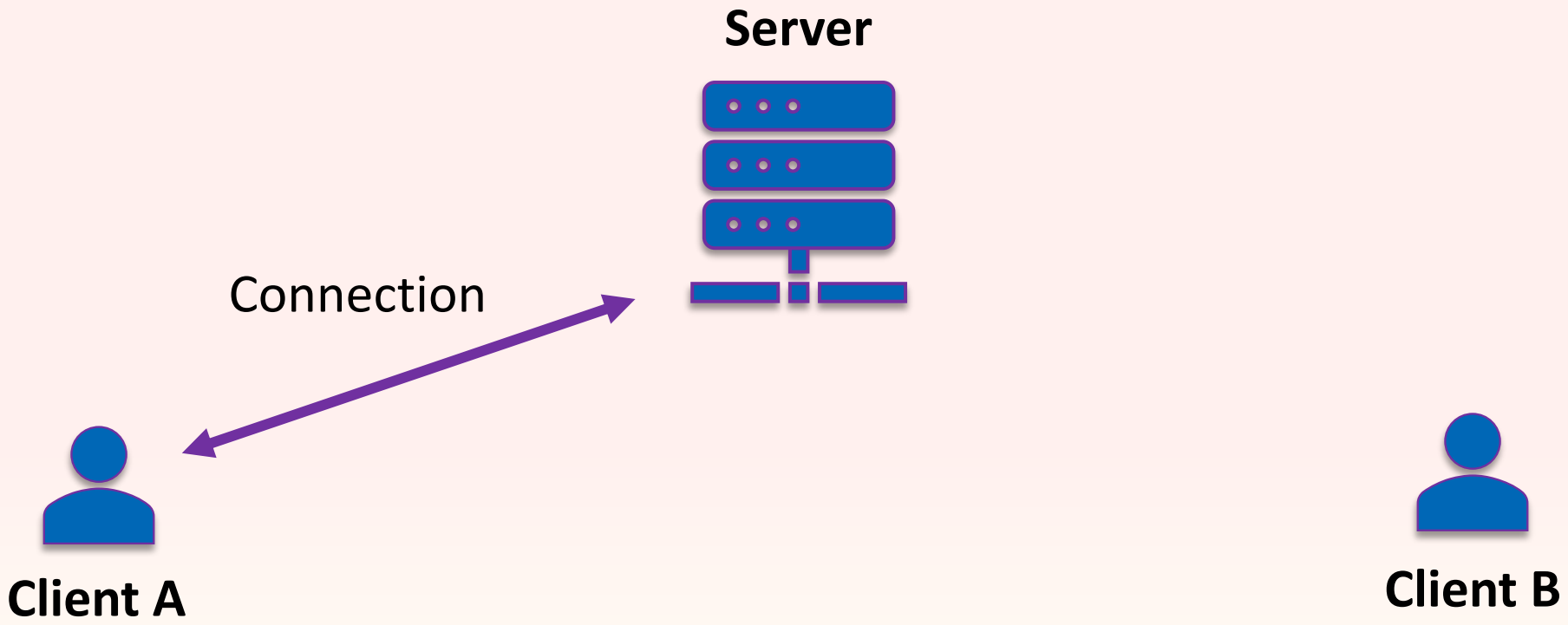
They were single-threaded and could only deal with one request at a time.



Concurrency: Client Requests

Think back to the simple server sockets from last week.

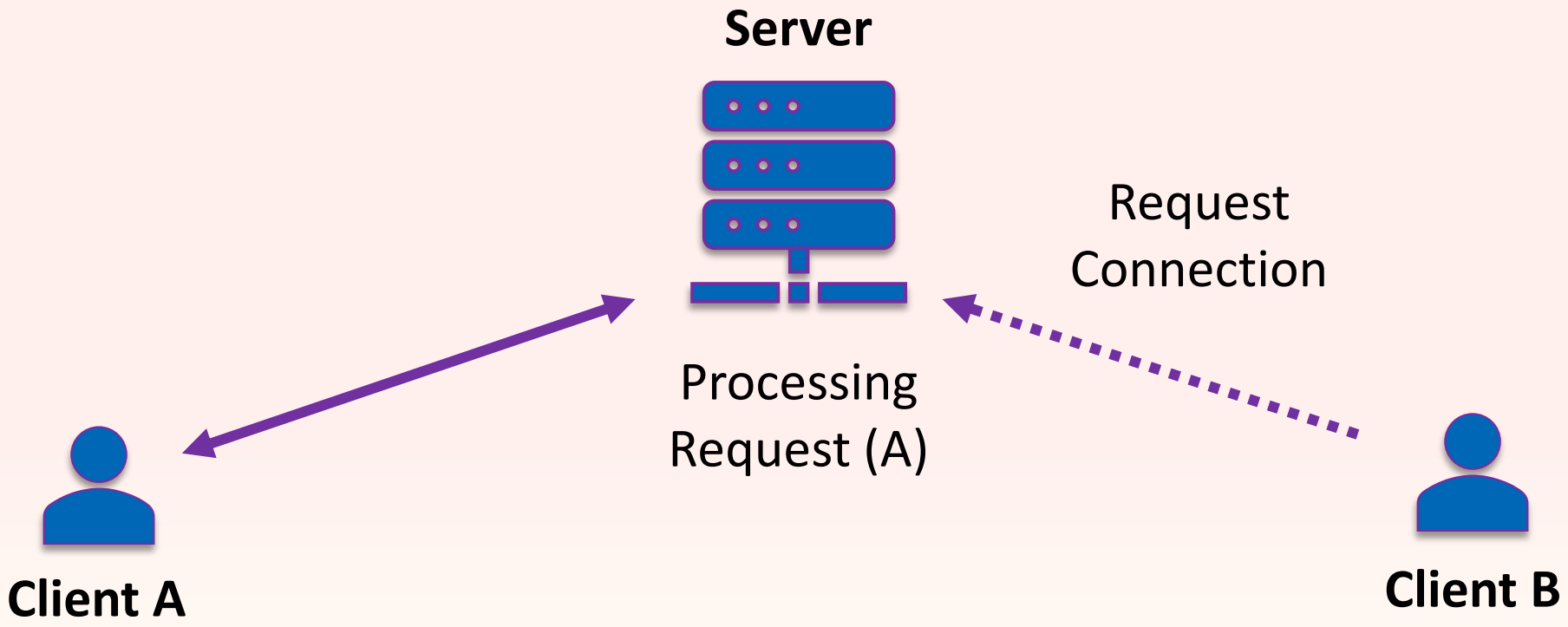
They were single-threaded and could only deal with one request at a time.



Concurrency: Client Requests

Think back to the simple server sockets from last week.

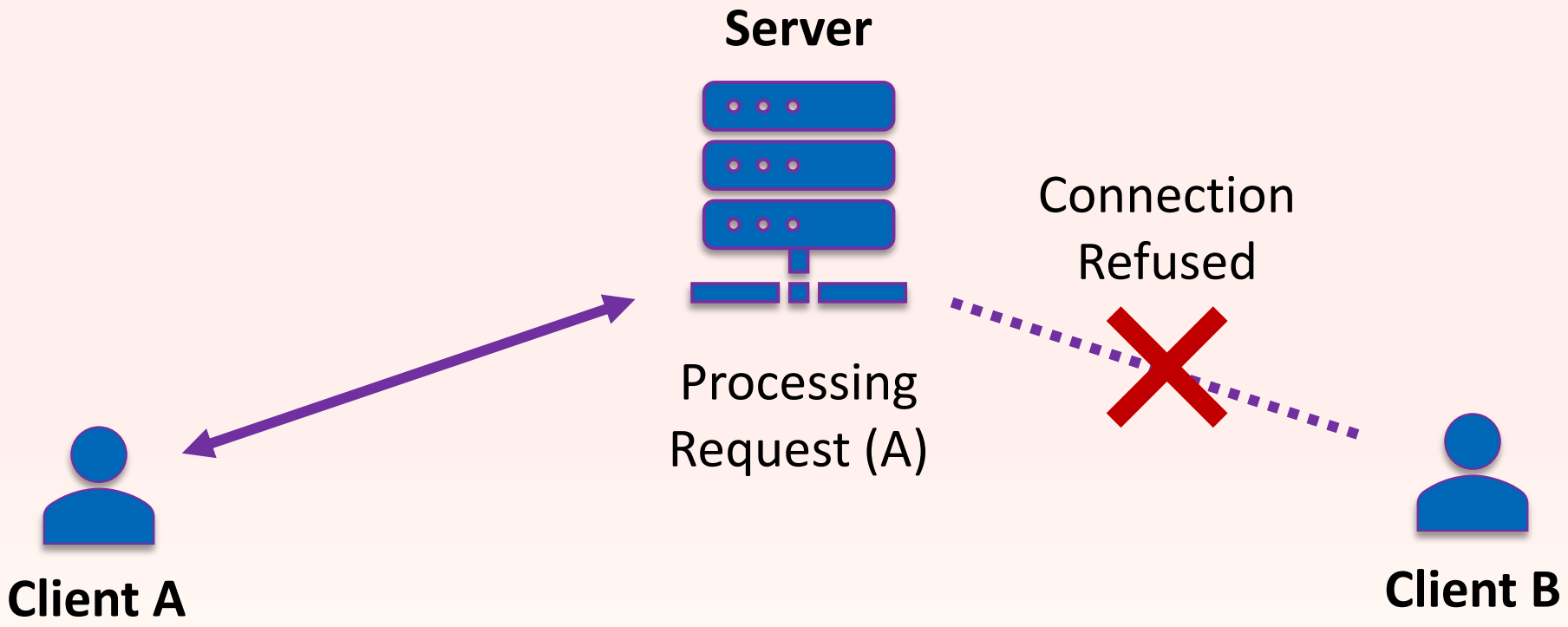
They were single-threaded and could only deal with one request at a time.



Concurrency: Client Requests

Think back to the simple server sockets from last week.

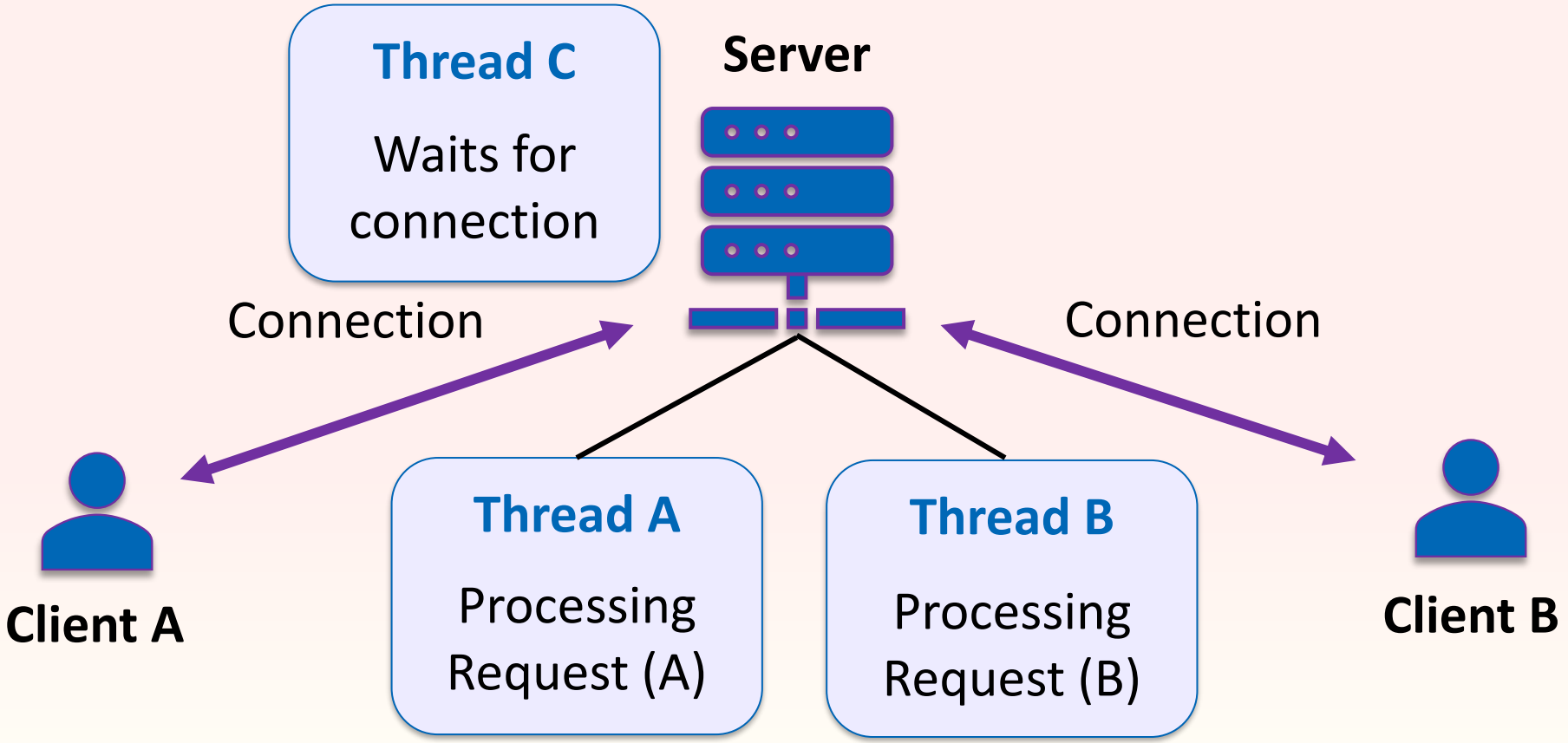
They were single-threaded and could only deal with one request at a time.



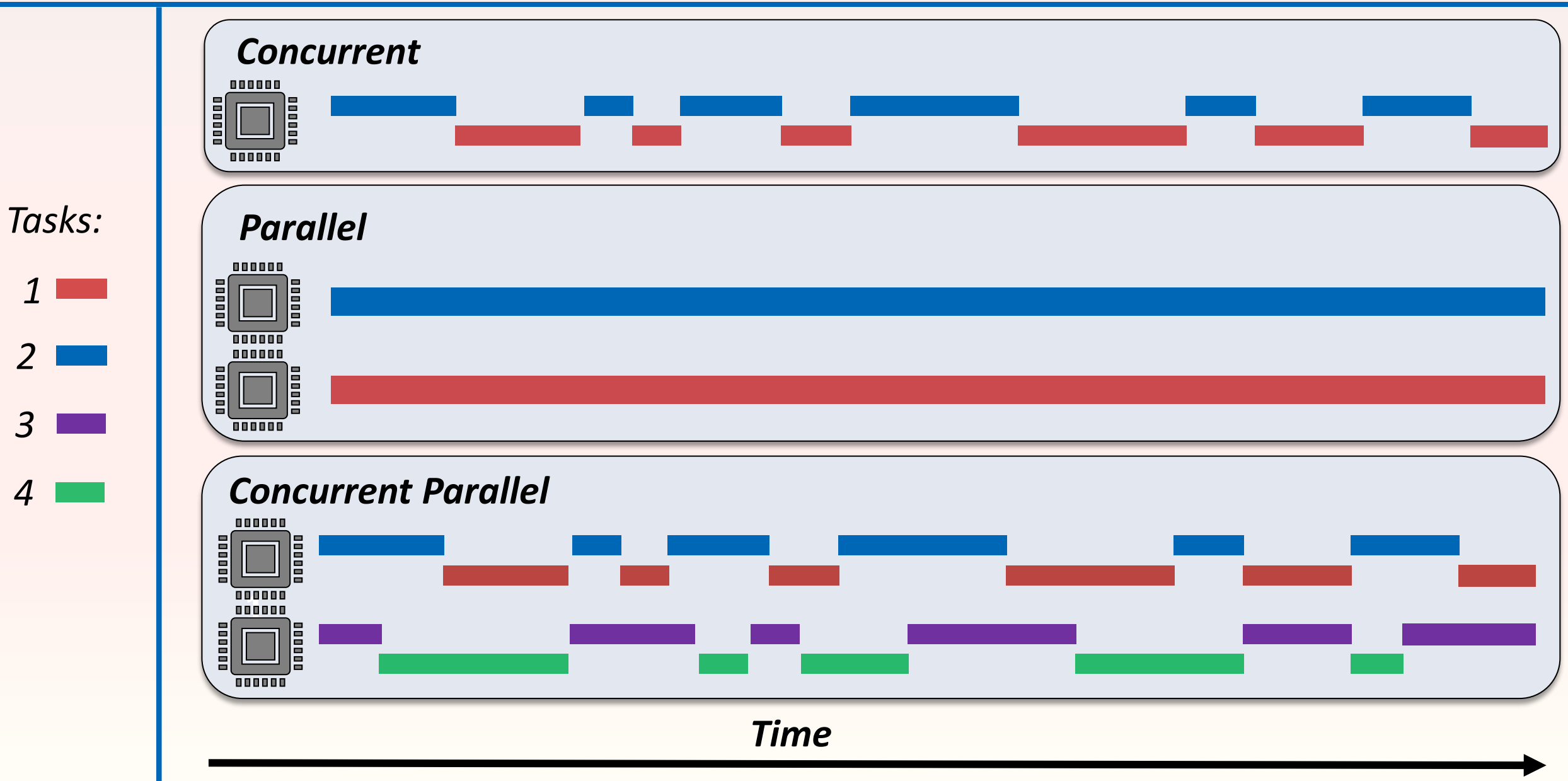
Concurrency: Client Requests - Multithreaded

If the server can spawn new threads as and when it needs them, it can both process existing requests *and* listen for new connections.

The number of clients and requests scales with the CPU cores of the server.



Concurrency: Not Parallelism



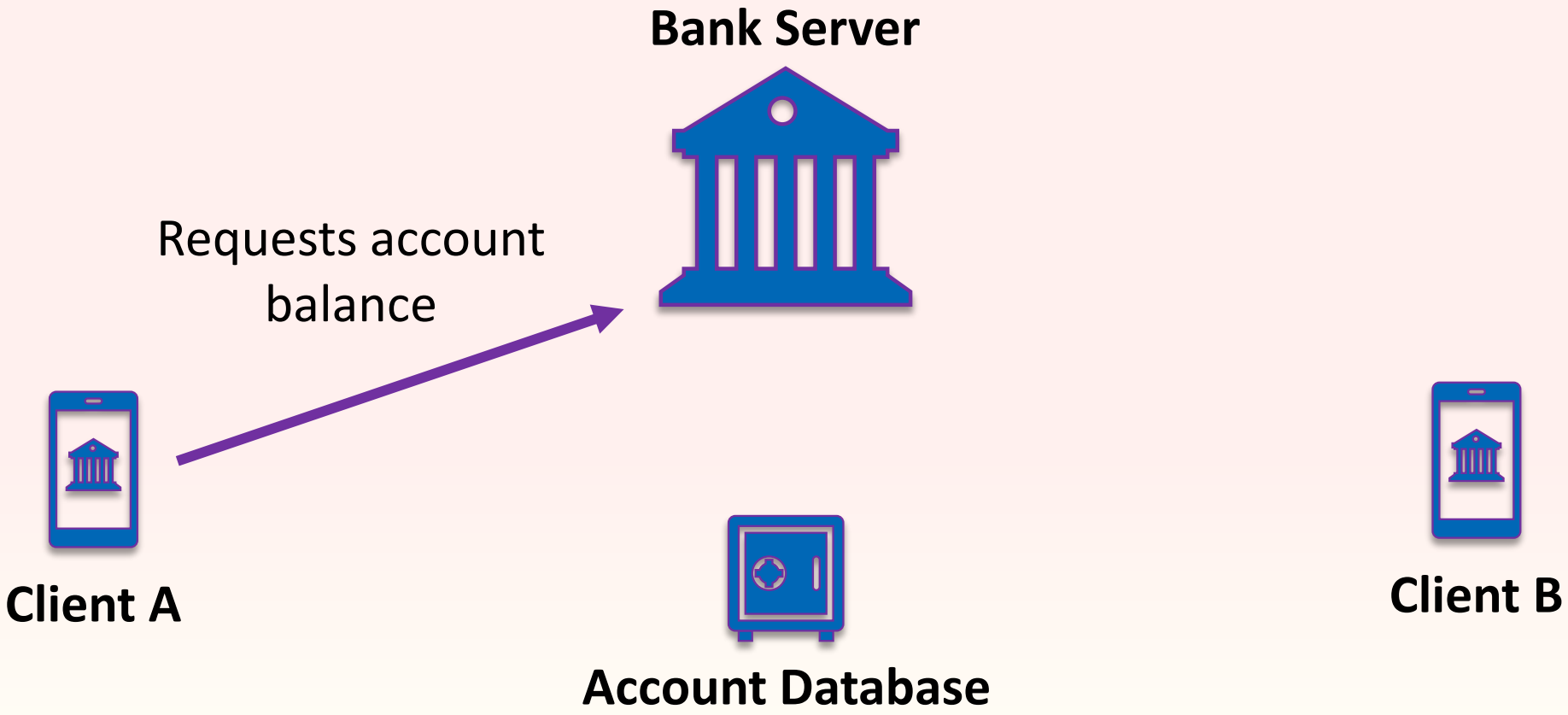
CPUs and Threads: Thread Lifecycle

State	Description

Concurrency: Readers and Writers

Consider the following:

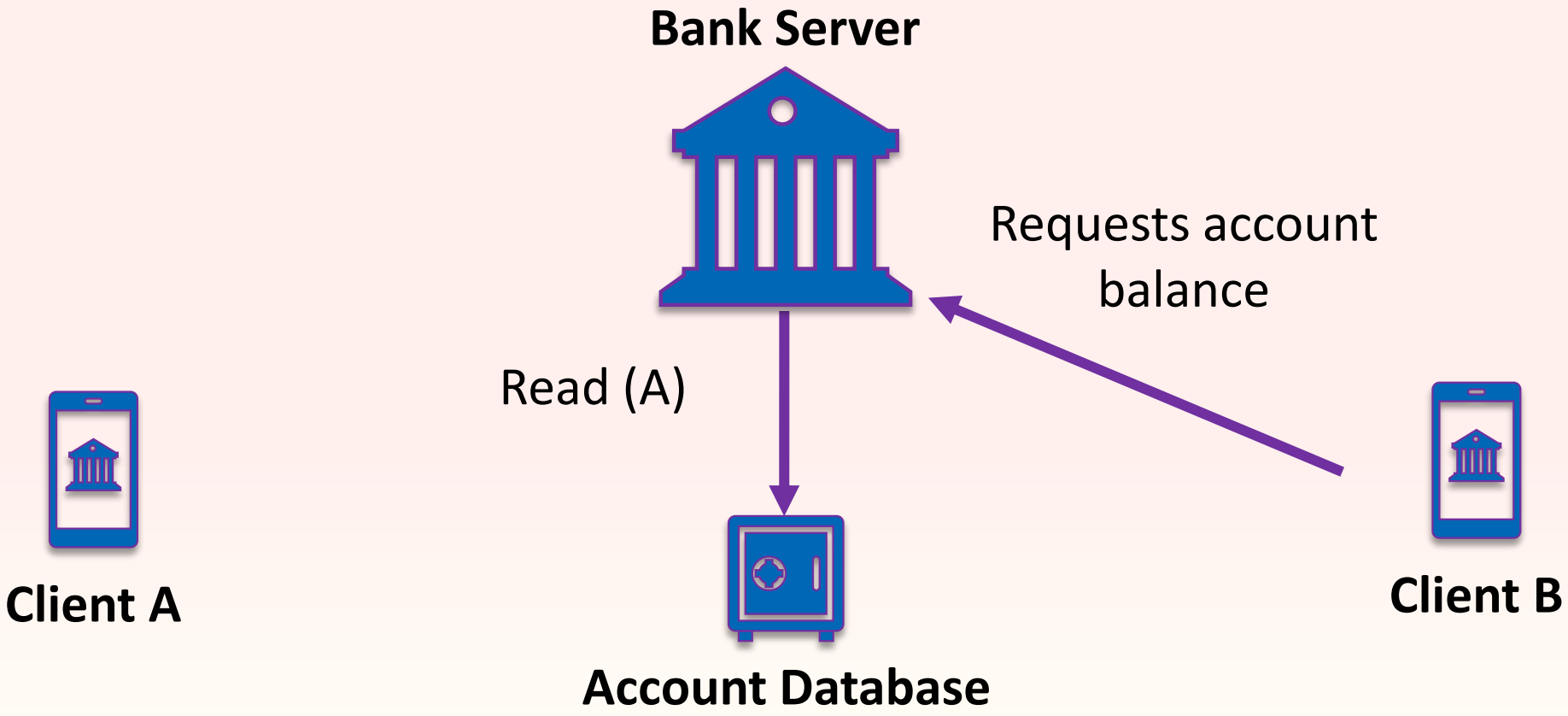
We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?



Concurrency: Readers and Writers

Consider the following:

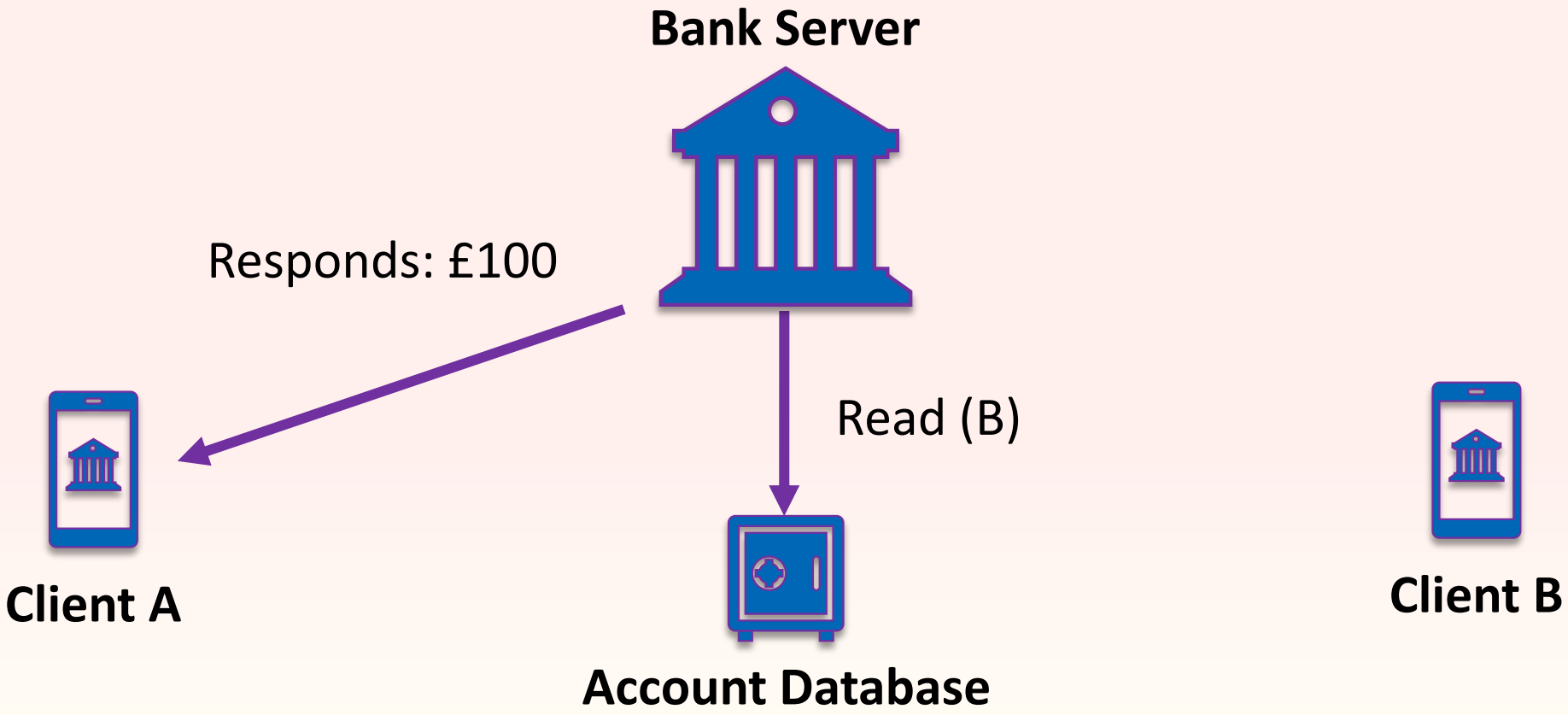
We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?



Concurrency: Readers and Writers

Consider the following:

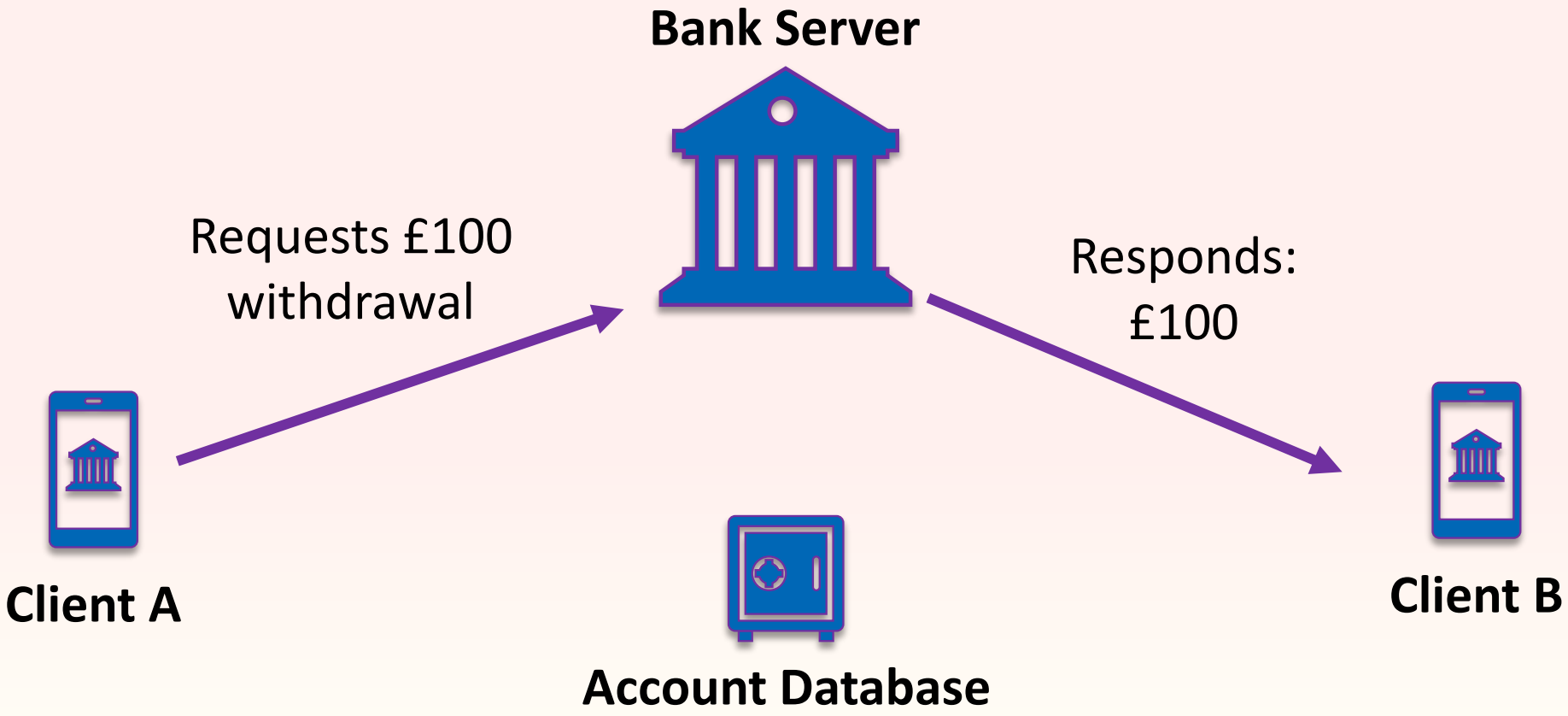
We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?



Concurrency: Readers and Writers

Consider the following:

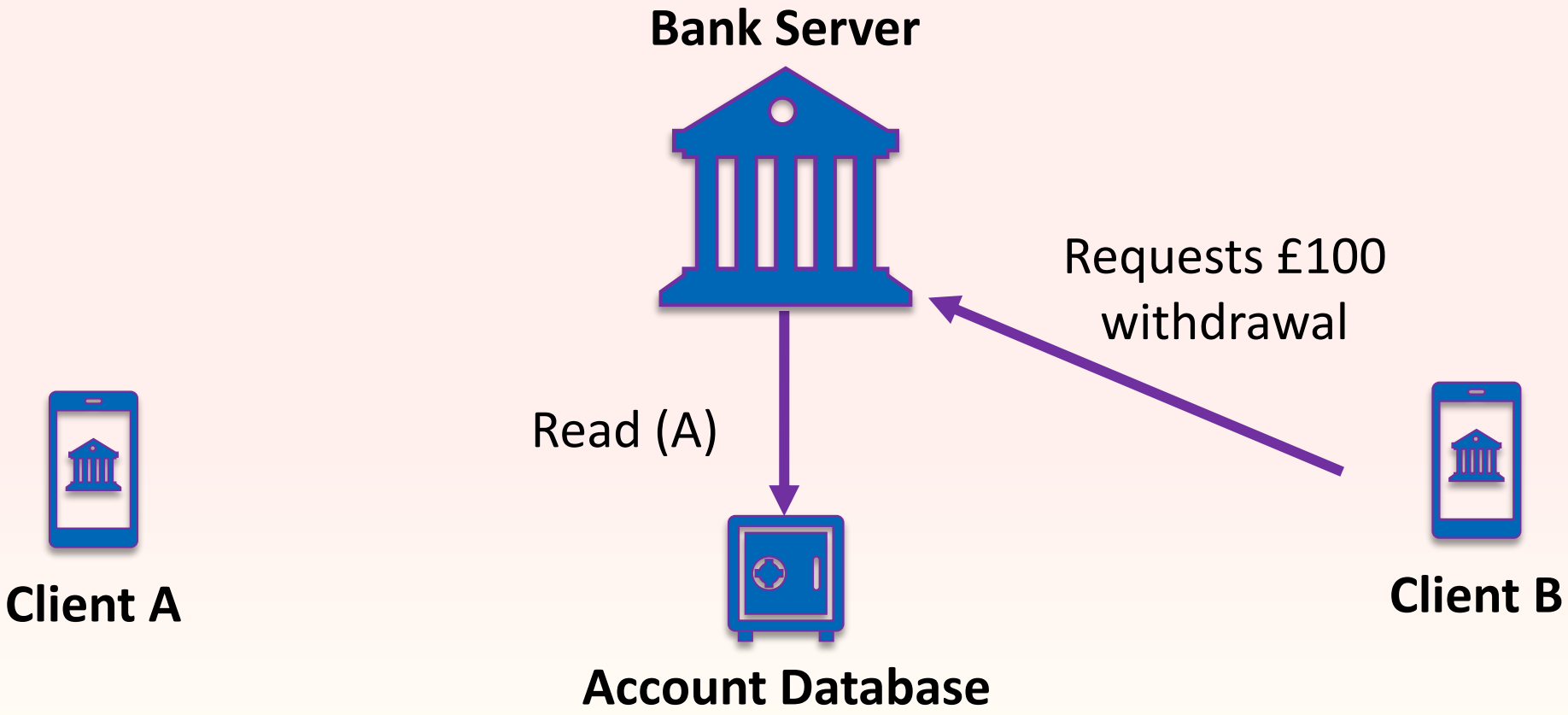
We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?



Concurrency: Readers and Writers

Consider the following:

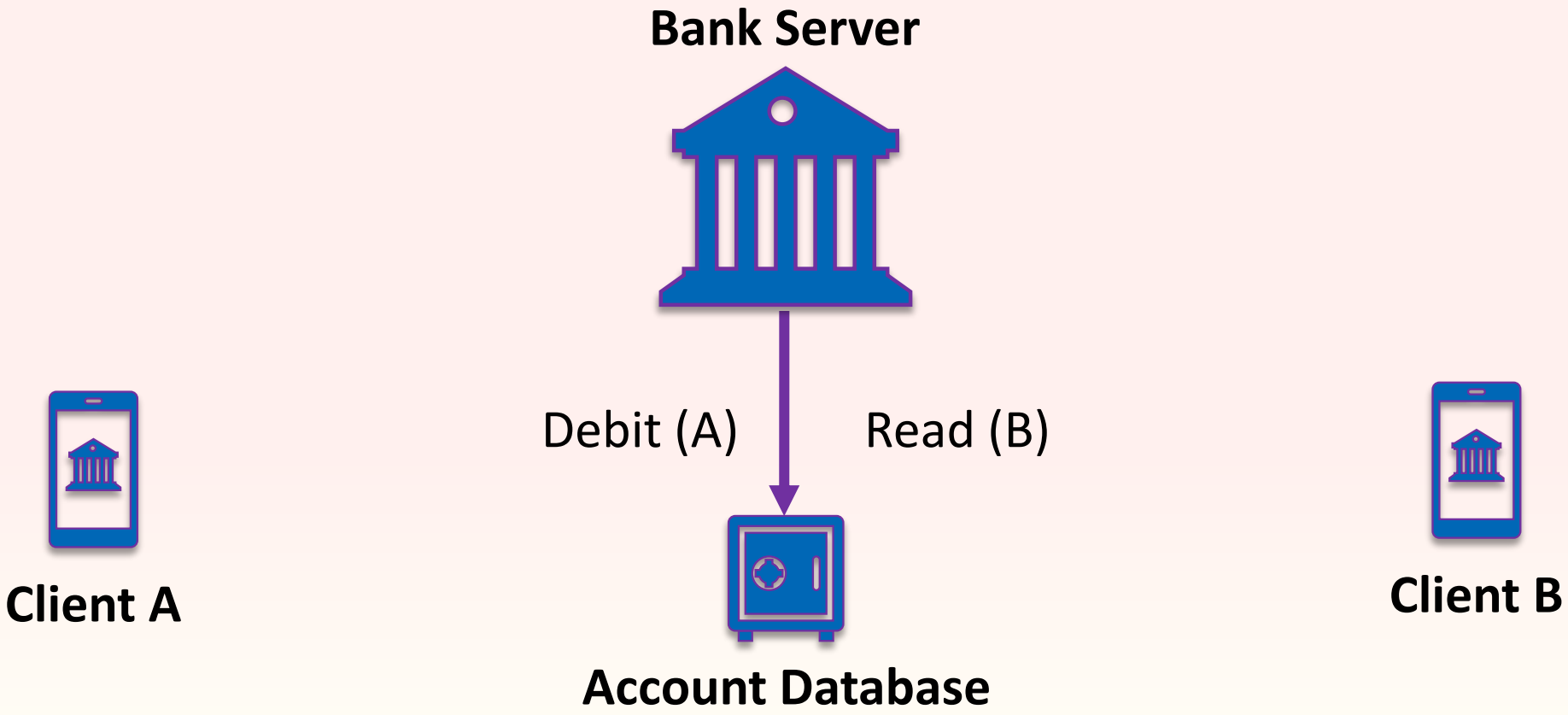
We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?



Concurrency: Readers and Writers

Consider the following:

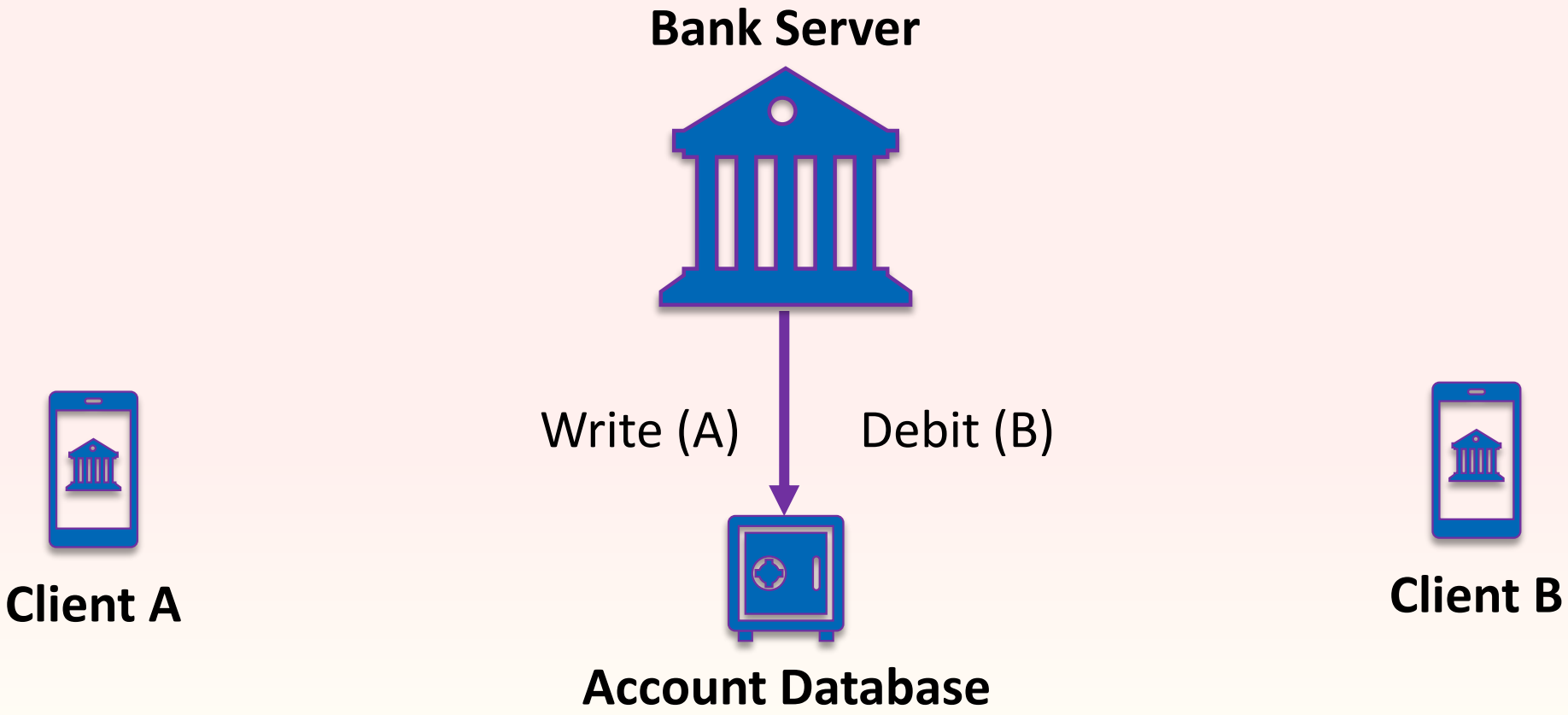
We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?



Concurrency: Readers and Writers

Consider the following:

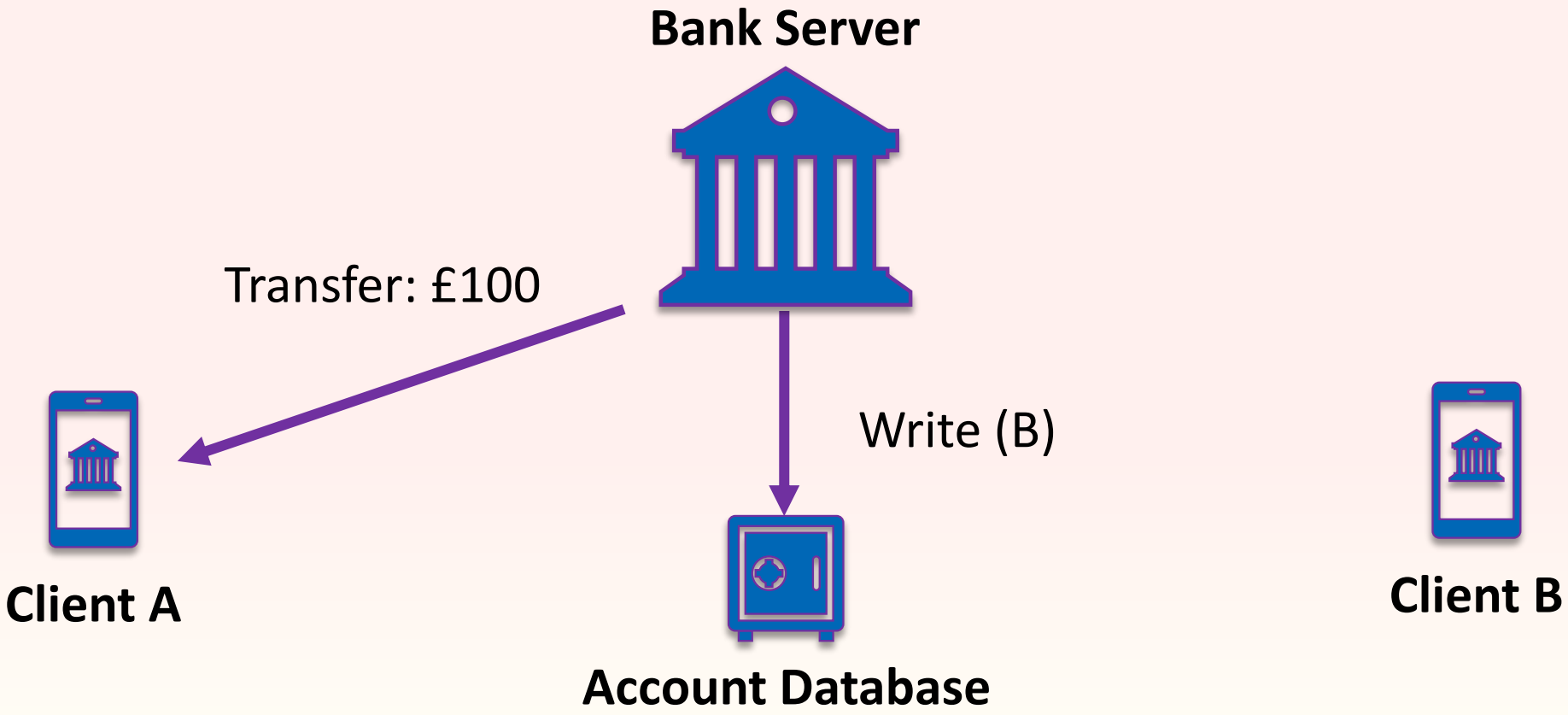
We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?



Concurrency: Readers and Writers

Consider the following:

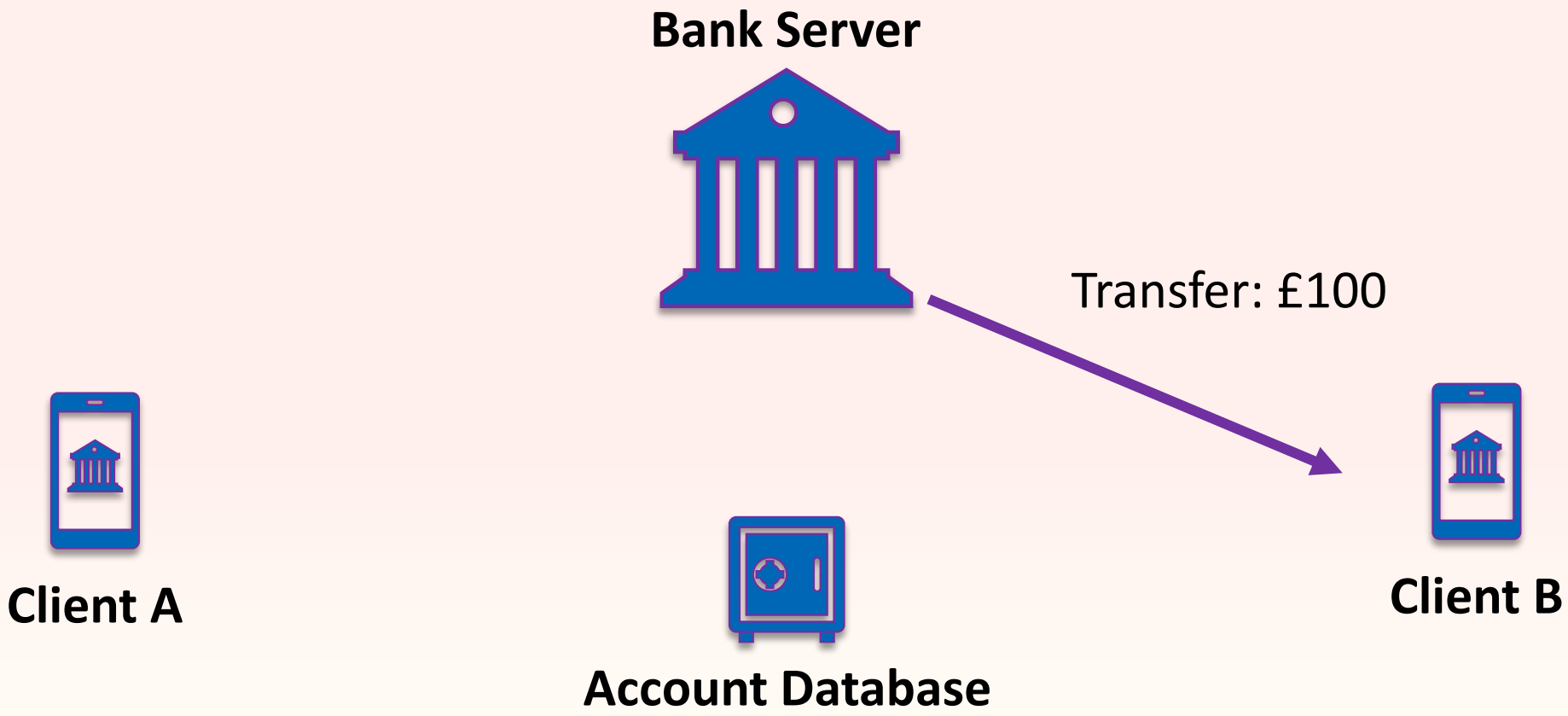
We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?



Concurrency: Readers and Writers

Consider the following:

We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?

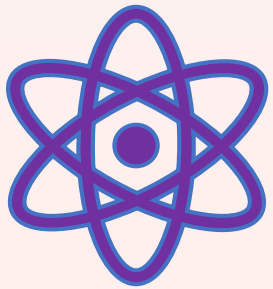


Concurrency: Key Takeaways

We need to pay attention to where operations:

- Read data
- Write data

And understand the difference between them.



Concurrent writes cannot be allowed.

- We can only allow one at a time.

Concurrent reads can happen, but:

- They cannot happen during a write.

Critical operations, when started, should not be interrupted!

In other words, they should be atomic

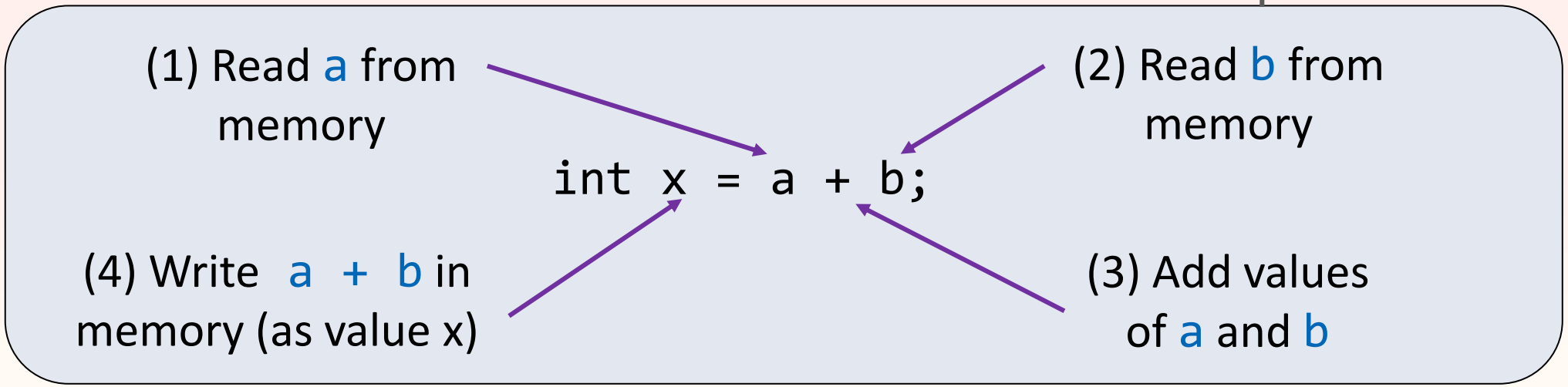
Concurrency: Atomics and Programming

Consider this Java snippet. Specifically, the third statement.

```
int a = 10;
int b = 5;
int x = a + b;
```

A thread managing this task can be interrupted by another thread running within the same process!

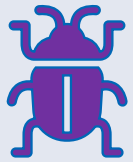
How many operations are happening here?



Concurrency: What could go wrong?

Application Errors

- Software doesn't do what you want it to do
- Programs execute in an 'unexpected' manner.
- Ultimately, the 'solution' is wrong.



Deadlock

- Multiple workers hold a resource needed by others.
- No one will release the resource until the other becomes available.
- Ultimately, everything stops.



Lockout

- Workers access resources to complete tasks with no queuing.
- Those workers indefinitely prevent another worker from accessing a resource it needs.
- It gets 'locked out'



Concurrency: So, what now?

Concurrency can lead to some horrific errors when things go wrong.

Those errors can be extremely difficult to debug and reproduce.

You cannot predict when a thread will finish one operations and move on to the next.

Fortunately, this isn't a new problem.

- Most programming languages have native support for writing concurrent software and controlling threads.
- But as you can imagine, the onus is on the programmer to avoid shooting themselves (and others) in the foot.

Threads in Java: Classes and Interfaces

Java has the Thread class, that (predictably) represents a thread.

The Thread class is-a Runnable (i.e., implements the interface called Runnable)

Thread thus inherits the method `run()` from Runnable which is where we put code the thread needs to run.

To create a thread of your own you can either:

- Create a subclass of Thread, override `run()`, and put your code in there, OR
- Create a class, make it implement Runnable, override `run()` with your code, and pass an instance of your class to a Thread object's constructor.

Both have their benefits and drawbacks.

Threads in Java: Extending Thread - Run

```
class MyThread extends Thread {
@Override
public void run() {
 for (int i = 1; i <= 5; i++) {
   System.out.println(this.getName() + " - Task: " + i);
   try {
       // Do some work
   } catch (InterruptedException e) {e.printStackTrace(); }
   }
 }
 }
```

Threads in Java: Extending Thread - Main

```
public class Main {
 public static void main(String[] args) {
   MyThread th1 = new MyThread();
   MyThread th2 = new MyThread();
   th1.start();
   th2.start();
   }
 }
```

Threads in Java: Implementing Runnable - Run

```
class MyTask implements Runnable {

@Override

public void run() {

   for (int i = 1; i <= 5; i++) {

       try {

              // Do some work

       } catch (InterruptedException e) {

              e.printStackTrace();

      }
       }

   }
 }
```

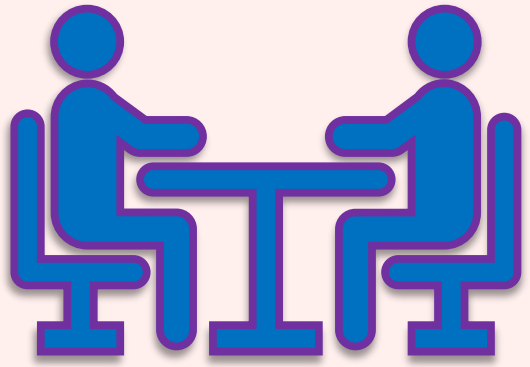
Threads in Java: Implementing Runnable - Main

```
public class Main {
 public static void main(String[] args) {
   MyTask task = new MyTask();
   Thread th1 = new Thread(task);
   Thread th2 = new Thread(task);
   th1.start();
   th2.start();
   }
}
```

Note:
Our class implementing Runnable is now passed to the Thread class constructor.

Summary: Part 1

- Modern hardware makes concurrent and parallel programming a reality for most programmers
- Controlling multiple threads to complete multiple tasks can mean improved performance in our applications
- With great power comes great cliches, and with many threads comes the responsibility to avoid race conditions!



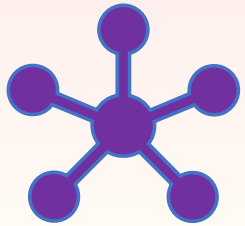
Next lecture: Multithreading in Java

DEMO

(Unsafe) Multithreading in Java

PART 2

Multithreading and Synchronisation



Multithreading: Monitors and Locks

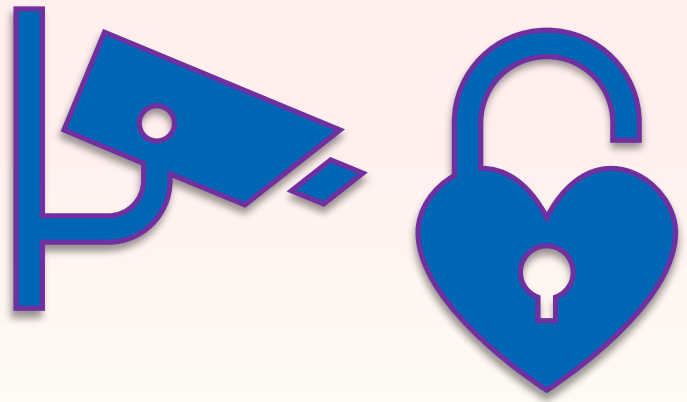
Okay, so we can cause race conditions in Java. Yay!

As mentioned last lecture, many languages introduce mechanisms to help prevent them.

In Java there are quite a few ways of controlling threads.

We're going to look at two lower-level approaches. These are:

- Locks
- Monitors



Multithreading: Monitors and Synchronisation

Generally, monitors are somewhat of an abstract notion.

Naturally, that means there are different implementations of them.

As far as we're concerned, they:

- Ensure only one thread can execute a specific block of code at a time.
- Use condition variables to control certain stages of a thread's life cycle.

Java supports monitors via the:

- `synchronized` keyword
- `wait()`, `notify()` and `notifyAll()` methods

Multithreading: synchronized Methods

```
public class synchronized Counter {

      private int c = 0;
      public synchronized void inc() {
            c++;
      }
      public synchronized void dec() {
            c--;
      }
      public synchronized int value() {
            return c;
      }
}
```

The synchronized keyword can be used in method signatures.

If multiple threads can access this object in shared memory:

- Only one thread can call a synchronized method at a time.
- All other threads must wait for the 'current' thread to finish before they can execute any other synchronized code.
- In other words, they are mutually exclusive (mutex)

Multithreading: `synchronized` Methods

Synchronised methods are useful, but they:

Can lead to inefficiencies

- The entire object's lock is locked (all synchronized methods blocked).
- The 'critical' operations might only be a small part of the method

Only ensure mutual exclusion for a *shared* object, not for arbitrary data!

- We may have an `ArrayList` we want our thread objects to interact with
- Synchronising access to the list would mean making a new wrapper class to control data manipulation through synchronised methods.
- That can be both inefficient and awkward.

Multithreading: synchronized Blocks

```
List<Socket> clients = new
                    ArrayList<>();

synchronized (clients) {
        clients.add(clientSocket)
}

synchronized (clients) {
        clients.remove(clientSocket)
}

synchronized (clients) {
       System.out.println(
             "Connected: " + clients)
}
```

The `synchronized` keyword can be used to create blocks.

- The block is parameterized by an object.
- Any other `synchronized` block, also parameterised by the *same* object, becomes mutually exclusive.

Note: Using `clients` as your block parameter doesn't 'protect' it. This just serves as a 'common' object to tie collections of synch blocks mutually exclusive.

Multithreading: `synchronized` Blocks – Common Objects

```
Object lock = new Object();

List<Socket> clients = new
                    ArrayList<>();


synchronized (lock) {
      clients.add(clientSocket)
}

synchronized (lock) {
      clients.remove(clientSocket)
}
```

We can just as easily:

- Create a new object and call it `lock`
- Use it exclusively, as our `synchronized` parameter.

Once inside the synchronized block:

- perform the critical operations on the object of interest.

`lock` will then be our reference for synchronising operations on the list of clients

Multithreading: `synchronized` Blocks - Gotchas

```
List<Socket> list1 = new ArrayList<>();
List<Socket> list2 = new ArrayList<>();

synchronized (list1) {
      if (list2.size() > 1000) {
            list1.add(clientSocket)
      }
}

synchronized (list2) {
    if (list1.isEmpty()) {
            list2.remove(clientSocket)
      }
}
```

Consider this example:

- We have two lists whose operations rely on each other.

What might be the problem?

- Their synchronsiation blocks do not share a common object!
- Operations on `list2` depend on the contents of `list1`
- But their blocks are not mutually exclusive!

Multithreading: Back to Readers and Writers

So, we can synchronise write operations to prevent race conditions.

But what about the reads? Ideally, we want to provide concurrent reads.

However, they should not happen *during* a write operation.

We could synchronise the reads too, but that would be wasteful.

- The number of reads often outweighs the number of writes.
- If we leave them alone, we could be reading incorrect information.

How might we solve this?

- Track the number of reads



Multithreading: Wait!

Let's say we have a counter class with synchronised methods to track when a read operation is happening.

We can use this inside synchronized blocks to determine when it's safe to perform a write operation.

```
Object lock = new Object();

synchronized (lock) {
    while (counter.getCounter() > 0) {
            lock.wait();
      }
      // Perform write operation
}
```

Implemented by the Object class

Effectively everything inherits from Object.

It forces the current thread to release the synchronous lock for now.



Multithreading: Wait!

Check if our 'thread-safe' read counter is non-zero

Once reads are done, perform the write.

```
Object lock = new Object();

synchronized (lock) {
    while (counter.getCounter() > 0) {
            lock.wait();
    }
    // Perform write operation
}
```

Wait if another synchronized block is occupied

If it is, give up the synchronous lock and wait for the last of the reads to finish.

Concept Check: The Hanging Thread

How does a sleeping thread know when the counter reaches zero?





Multithreading: Threads, Asseeeeeemble!

Something needs to wake up threads that call `wait()`.

These are the methods for this, and they should be called in a logical place!

`notify()`: Wakes up a single thread waiting on an object's monitor

`notifyAll()`: Wakes up all threads waiting on an object's monitor

```
public class Counter {
        public synchronized void decrement() {
                c--;
                if (c == 0) {
                        clients.notifyAll();
                }
        }
}
```

Multithreading: Summarising Readers and Writer

Read operations need to increment and decrement counters using synchronised methods because:

- Write operations might already be happening.
- The counter will need to wait for the write to finish. Once that happens, it can increment the counter.
- Once the counter change starts, no write operations can go ahead and will have to wait.
- Counters decrement when finishing a read, preventing write operations until notify is called and briefly preventing other read operations from changing the counter.
- If a write operation starts, then it can only enter the synchronized block if no other threads are also trying to write or there are ‘active’ reads.



Multithreading: A simpler Approach

Recall that monitors are one of many ways to control threads.

Another is Locks

The `java.util.concurrent` API offers a number of richer, more flexible alternatives to the implicit monitor locks used by synchronization primitives.

Java provides Lock libraries, as a layer of abstraction from the we just looked at.

As was the case with I/O streams, there are many different types of locks.

The two types we'll concern ourselves with are:

- `ReentrantLock`
- `ReadWriteLock`



Multithreading: Lock at the Simplicity

```
ReadWriteLock lock = new ReentrantReadWriteLock();
```

```
lock.readLock().lock();
```

Other code can enter this section, or other sections protected by this readLock **provided** no other code is in the corresponding writeLock section.

```
lock.readLock().unlock();
 ...

lock.writeLock().lock();
```

Nothing else can happen here - no other writes, and no reads.

```
lock.writeLock().unlock();
```



Multithreading: Read Write Locks

The ReadWriteLock offers a much simpler (and perfectly acceptable) solution to the read write problem we solved using synchronized blocks and the counter.

```
readWriteLock.writeLock().lock();

  if (some condition) {

     // Do write operation
  }

readWriteLock.writeLock().unlock();
```

```
readWriteLock.readLock().lock();

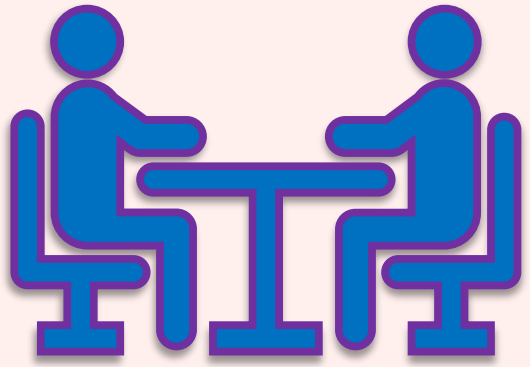
  if (some condition) {

     // Do read operation
}

readWriteLock.readLock().unlock();
```

Summary: Part 2

- Java has a quite a few approaches to controlling threads – some more complicated than others.
- You can synchronize methods, blocks of code or opt to use special lock objects from the concurrent API
- There is no 'right' way, but there are many, *many* wrong ways!



Lab: Java Threads

Next lecture: Web Services and HTTP